

One Tensor Commitment for Tables of Every Height

Andrei Kochergin
Oumuamua Labs
andrei@oumuamua.dev

Abstract

We commit every table of a multi-table AIR proof in one tensor commitment: one Merkle tree per oracle whose leaf hashes every table’s part of a codeword column, one fold vector, one set of t opened columns, and one sumcheck that reduces every table’s evaluation claims to a shared point. Committed separately, T tables pay T fold vectors and T query sets of $t = 287$ columns each. A table of 2^n rows sits on a grid of 2^c columns: a shorter table packs 2^{c-n} of its columns into one message row, a taller one keeps 2^{n-c} grid rows, and zero extension to a common height makes the pool a single table, whose proximity error is the single-table error with $\Sigma + m - 1$ batched units for Σ slot roles and m masters. Four rules carry the construction, and for each we give the forgery or the inconsistency its violation admits. The split c minimizes an integer byte count that charges the blind claims of packed tables, and both parties derive it from absorbed data. Merkle leaves without salts need $t + 1$ random support cells per row: at t cells every unopened leaf of a tree without blind rows is a function of the witness and the verifier’s view, and at $t + 1$ every unopened symbol is uniform given that view. In Hekate’s zero-knowledge mode, against separate commitments, proof size changes by -36 to -56% on ML-KEM-768, ML-DSA and AES and by -88% on RSA-2048.

1 Introduction

A proof over T AIR tables can open all of them with one fold vector and one set of queried columns. We commit every table of a proof in one Merkle tree per oracle and reduce every table’s evaluation claims with one sumcheck.

The setting is a tensor polynomial commitment in the style of Ligero and Brakedown [1, 7] over the binary tower $\mathbb{F}_{2^{128}}$ [3], with a Reed–Solomon row code evaluated by the additive FFT [11] and claims on bit-packed columns reduced by ring-switching [5]. Each table carries its own ZeroCheck, and LogUp buses [8] connect the tables. Every table then owes evaluation claims on its committed columns at points of its own height.

Committed separately, table i pays a Merkle root, a fold vector of $S + 2^{c_i}$ field elements for its own split $c_i \leq n_i$, and t opened columns with their Merkle multiproof. For the seven tables of an ML-DSA-65 verification this layout produced a zero-knowledge proof of 2 959 KiB.

In the pool, a split c fixes one grid of 2^c columns for every table. A table of 2^n rows with $n \geq c$ keeps 2^{n-c} grid rows; a table with $n < c$ packs 2^{c-n} of its columns side by side into one message row, and the $c - n$ bits that select a column become variables of the evaluation sumcheck. One leaf hashes every table’s digest of one codeword column. One sumcheck over $v = \max(c, \max_i n_i)$ variables carries every table, a table with fewer variables entering the remaining rounds through the factor $\text{eq}(z, 0)$. After zero extension the pool is one table of 2^v rows whose slots vanish above their table’s height (Lemma 2), and its proximity error is the single-table error on that shape (Section 5).

Four rules carry the construction (Section 4): the slot challenge η follows every table's claims, every master carries its own power of the line challenge λ , a finished table is extended by $\text{eq}(z, 0)$ and never by a constant, and a short table's block bits are sumcheck variables. The split minimizes an integer byte count (Section 6). Leaf hiding needs one support cell more than the query count (Section 7): at $S = t$ an unopened leaf of a tree without blind rows is a function of the witness, which a guess can test against the root.

In Hekate's zero-knowledge mode the pool changes proof size by -36 to -56% on ML-KEM-768, ML-DSA and AES and by -88% on RSA-2048 (Section 8).

Batching polynomials of one length under one commitment is the interleaved setting of Liger and Brakedown, and the proximity bounds are those of [2] and [4]. Matrices of differing heights are committed under one Merkle tree in FRI stacks [13], and claims on columns of differing heights are reduced inside the sumcheck by a jagged reduction [9, 6]. Here the heights are resolved in the layout: packing and zero extension make the pool one table, and one fold vector, one query set and one proximity term cover every table. New here are the layout for tables of different heights under one fold vector, the four rules with the attacks their violations admit, the split objective, the support requirement for unsalted leaves, and the measurements. The per-table ZeroCheck and LogUp arguments are unchanged and outside this note. The outer argument of [10], which proves the verifier's predicate on one-time-padded scalars, runs only in zero-knowledge mode, and we assume its soundness and zero-knowledge.

2 Notation and the row code

Throughout, $\mathbb{F} = \mathbb{F}_{2^{128}}$, built as a tower of quadratic extensions [3], and $|\mathbb{F}| = 2^{128}$. For $a, x \in \mathbb{F}^n$ put $\text{eq}(a, x) = \prod_j (a_j x_j + (1 + a_j)(1 + x_j))$. A table of 2^n rows commits each column f as the multilinear polynomial in n variables that agrees with f on $\{0, 1\}^n$, and $\tilde{f}(a) = \sum_{x \in \{0, 1\}^n} \text{eq}(a, x) f(x)$ is its value at a .

Fix a split c and put $L = 2^c$. A message is a pair (ξ, d) with a support $\xi \in \mathbb{F}^S$, uniform in zero-knowledge mode, and data $d \in \mathbb{F}^L$. Its codeword is the polynomial $\sum_{k < S} \xi_k X_k + \sum_{k < L} d_k X_{S+k}$ evaluated on an $\mathbb{F}_2$ -subspace of W points, where $X_0, X_1, \dots$ is the novel polynomial basis of [11] over a Cantor basis, $\deg X_k = k$. The code has length $W = 2L$ when $S \leq L$ and the query term below reaches 100 bits at that length. At $S = 288$ and $t = 287$ the term reads 82 bits at $c = 10$ and 100.1 at $c = 11$: a grid with $c \leq 10$ takes L support cells at $W = 4L$, and every deployed split has $c \geq 12$. The row code is Reed–Solomon of dimension $S + L$ and length W , hence MDS, with minimum distance $W - S - L + 1$.

A table of 2^n rows with $n \geq c$ stores a column f on $R = 2^{n-c}$ grid rows: grid row $g \in \{0, 1\}^{n-c}$ holds $f(x, g)$ for $x \in \{0, 1\}^c$. The columns of a table are grouped into slots, one slot per column here, and each pair of a slot and a grid row is one message with its own support. The encoded messages form a matrix of W columns, and the commitment is a Merkle tree over those columns.

The verifier batches a table's claims into at most two master claims. Each unit u of the table, a committed column or a ring-switched group of bit columns, carries a power $\eta^{\sigma(u)}$ of a slot challenge η , and a slot's coefficient $\kappa_{k,s}$ in master k is the sum of the powers of its units assigned to k . Master k is $M_k = \sum_s \kappa_{k,s} D_s$, D_s the multilinear polynomial of slot s , and a sumcheck reduces the table's batched claims to the values $\mu_k = M_k(r)$ at one point r . The prover sends a fold vector

$$q = \sum_k \lambda^k \sum_s \kappa_{k,s} \sum_g \text{eq}(r_{\text{hi}}, g) (\xi_{s,g}, d_{s,g}),$$

where r_{hi} are the last $n - c$ coordinates of r and r_{lo} the first c . The verifier checks the line $\langle d_q, \text{eq}(r_{\text{lo}}, \cdot) \rangle = \sum_k \lambda^k \mu_k$ on the data part d_q of q , draws t columns j , and checks that the encoding of q at j equals the same combination of the opened symbols, with their Merkle paths.

The single-table analysis bounds two terms. The query term is $-t \log_2((1 + \rho)/2)$ bits with $\rho = (S + L)/W$ [7, App. B, Thm. 7]. The proximity error is $(\theta + U - 1)W/|\mathbb{F}|$, with $\theta = \log_2 R$ the depth of the tensor over grid rows [4] and $U - 1$ the degree of the batching curve over U units [2, Thm. 1.5]; Section 5 counts U for the pool.

3 The pool

The pool is every table of a proof in a fixed order, T tables with 2^{n_i} rows each. The split c is derived in Section 6; put $v = \max(c, \max_i n_i)$ and $v_i = \max(c, n_i)$. Table i uses $R_i = 2^{\max(n_i - c, 0)}$ grid rows and packs $P_i = 2^{\max(c - n_i, 0)}$ columns per slot.

A table with $n_i \geq c$ is laid out as in Section 2. A table with $n_i < c$ has one grid row, and its slot holds P_i columns side by side: column (s, b) for $b \in \{0, 1\}^{c - n_i}$ occupies positions $b \cdot 2^{n_i} + x$, and $D_s(x, b)$ is a multilinear polynomial in c variables. A class of columns fills $\lceil \text{count}/P_i \rceil$ slots, the last closed by zero blocks. In zero-knowledge mode table i adds $K P_i$ whole blind columns, $K = 1$ in the deployed system, and its ring-switched units a blind of their own. The slot roles of every table are numbered by one index σ running over the pool in table order, Σ in all, and the pool has m masters, one or two per table.

Every message of every table is encoded at the same width W . Leaf j of the trace tree is

$$H(0\text{x}02 \parallel \chi_{0,j} \parallel \cdots \parallel \chi_{T-1,j}), \quad \chi_{i,j} = H(0\text{x}00 \parallel \text{part}_{i,j}),$$

where $\text{part}_{i,j}$ is the column- j symbols of table i over its slots and grid rows, and an inner node hashes $0\text{x}01$ and its two children. The verifier hashes $T + 1$ times per opened column. The LogUp helper columns h of the bus-carrying tables form slots in a second tree of the same width, committed after the bus challenges; a pool with no bus has none. The fold vector and the queries cover both trees.

The transcript runs in this order.

1. The main table's dimensions, then each chiplet's in pool order, then the trace root; in zero-knowledge mode, the root of the one-time pad.
2. The LogUp challenges γ, β ; every lookup bus's maximal height, then each bus's point; then the root of the h tree.
3. For each table in pool order: its claimed bus sums, its ZeroCheck challenge and rounds, its h evaluations and its column claims.
4. The slot challenge η ; $\max_i(c - n_i)$ block challenges, of which table i reads the first $c - n_i$ as ρ_i ; the ring-switching challenge when a table carries ring units; the eval sumcheck; the masters $\tilde{\mu}_k$ in pool order; λ when $m > 1$; the fold vector q ; the t query columns; both openings.

For table i split $y \in \mathbb{F}^v$ as $y = (x, b, z)$ with $x \in \mathbb{F}^{n_i}$, $b \in \mathbb{F}^{v_i - n_i}$ and $z \in \mathbb{F}^{v - v_i}$; a tall table has no b , and a table with $v_i = v$ has no z . The eval sumcheck runs over v variables, low variable first, on

$$F(y) = \sum_i \text{eq}(z, 0) \text{eq}(\rho_i, b) \sum_{k \in K_i} \omega_k(x) M_k(x, b), \quad (1)$$

where K_i are the masters of table i and ω_k are public multilinear weights fixed by the table's claim points, η and the ring-switching challenge. Its initial claim is $\sum_i \tau_i$, with τ_i the batched value of table i 's claims, which the verifier computes from those claims. Each round polynomial has degree 2. At the final point $r = (r_x, r_b, r_z)$, read per table, the verifier checks

$$F(r) = \sum_i \text{eq}(r_z, 0) \text{eq}(\rho_i, r_b) \sum_{k \in K_i} \omega_k(r_x) \mu_k \quad (2)$$

against the masters the prover then sends, $\tilde{\mu}_k = \text{eq}(r_z, 0) \mu_k$ for the master k of table i with μ_k the claimed value of $M_k(r_x, r_b)$. Let $r_{\text{lo}}, r_{\text{hi}}$ be the first c and the last $v - c$ coordinates of r . The fold vector is

$$q = \sum_s \left(\sum_k \lambda^k \kappa_{k,s} \right) \sum_{g < R_{i(s)}} \text{eq}(r_{\text{hi}}, g) (\xi_{s,g}, d_{s,g}), \quad (3)$$

with g read as a point of $\{0, 1\}^{v-c}$, and the verifier checks the line

$$\langle d_q, \text{eq}(r_{\text{lo}}, \cdot) \rangle = \sum_k \lambda^k \tilde{\mu}_k \quad (4)$$

and, at each of the t drawn columns j , that the encoding of q equals the same combination (3) of the opened symbols of both trees.

Lemma 1. *For an honest prover, (4) holds.*

Proof. It suffices to show $\sum_{g < R_i} \text{eq}(r_{\text{hi}}, g) \langle d_{s,g}, \text{eq}(r_{\text{lo}}, \cdot) \rangle = \text{eq}(r_z, 0) D_s(r_x, r_b)$ for each slot s of table i . If $n_i \geq c$, then $r_{\text{lo}} = r_x$ restricted to its first c coordinates, and for $g < R_i$ the factor $\text{eq}(r_{\text{hi}}, g)$ splits as $\text{eq}(r', g) \text{eq}(r_z, 0)$, r' the coordinates $c, \dots, n_i - 1$ of r ; the sum over g and the inner product together evaluate D_s at r_x . If $n_i < c$, then $R_i = 1$, $r_{\text{lo}} = (r_x, r_b)$ and $\text{eq}(r_{\text{hi}}, 0) = \text{eq}(r_z, 0)$, and the inner product is $D_s(r_x, r_b)$ by the layout of packed columns. Summing over slots with the coefficients of (3) gives $\sum_k \lambda^k \tilde{\mu}_k$. $\square$

4 Four rules

Each rule below is necessary. An interactive simulation over $\mathbb{F}$ with the deployed additive FFT, on four tables of 2^8 , 2^7 , 2^4 and 2^3 rows at $c = 6$, $S = 16$, $W = 512$ and 24 queries, exhibits the forgery or the inconsistency of each violation. The verifier of the final build rejected the forging provers of the first two rules in both modes.

1. The slot challenge η follows the claims of every table. A prover that knows η before it fixes the claims of one table moves two of them by δ_1, δ_2 with $\delta_1 \eta^{a_1} + \delta_2 \eta^{a_2} = 0$. The batched value τ_i is unchanged and every later check passes. The forging prover that drew η from a copy of the transcript ahead of the last table's claims was rejected, at the sumcheck in base mode and at the outer argument's linear test in zero-knowledge mode, because the verifier draws η after those claims.
2. Every master carries its own power λ^k . Under one pair $(1, \lambda)$ for the whole pool, a whole master of table i and one of table i' enter (4) with the same coefficient, and one claim error routed through the sumcheck into both masters cancels there in characteristic 2. The forging prover carried the drift through the eval rounds and moved the first and the last whole master; the verifier rejected it at the line.

3. A table with $v_i < v$ enters the rounds beyond v_i through $\text{eq}(z, 0)$. Carried as a constant a instead, it contributes $\sum_{z \in \{0,1\}^{v-v_i}} a = 2^{v-v_i} a = 0$ to the initial claim: in characteristic 2 every shorter table drops out, and its claims go unchecked. In the simulation a forged claim on the 2^3 -row table left the target of the constant-padded sumcheck unchanged.
4. The block bits of a short table are sumcheck variables. Opened at its own point (r_x, ρ_i) , a short table's master is another linear form on its slot than the one (4) applies to q , and the line fails for an honest prover. In the simulation the two points disagreed on one slot.

The root of the h tree precedes the first ZeroCheck challenge of every table, the rule a single table already obeys, held across the pool.

5 Soundness of the pool

To the verifier the pool is one table, and its proximity error is the single-table error with $U = \Sigma + m - 1$. Let $\mathcal{P}$ be the matrix with one row for each pair (s, g) of a slot of the pool and a point $g \in \{0, 1\}^{v-c}$, equal to the message of slot s on grid row g when $g < R_{i(s)}$ and to zero otherwise.

Lemma 2. *On the line and the column checks, the verifier of the pool runs the verifier of one table on $\mathcal{P}$, with grid depth $\theta = v - c$ and fold weights $(\sum_k \lambda^k \kappa_{k,s}) \text{eq}(r_{\text{hi}}, g)$, and every row of $\mathcal{P}$ outside the commitment is zero in every column.*

Proof. A zero message encodes to the zero codeword. Summing the opened symbols over the committed rows, as the column check does, therefore equals summing over every row of $\mathcal{P}$. The fold vector (3) is the fold of $\mathcal{P}$ by the stated weights, and by Lemma 1 the line (4) is the line of that table at r with the zero-extended masters $\tilde{\mu}_k$. $\square$

Write the fold of the encoded rows of $\mathcal{P}$ as $\sum_k \lambda^k c_k$, c_k the fold with the weights $\kappa_{k,s} \text{eq}(r_{\text{hi}}, g)$ of master k , and fix δ below the unique-decoding radius. The tuple $(c_k)_k$ is a word of the m -fold interleaved code, combined from the rows of $\mathcal{P}$ along a curve of degree $\Sigma - 1$ in η , the slot roles carrying distinct powers, and the tensor over 2^θ grid rows. If $\mathcal{P}$ is δ -far from the code, [2, Thm. 1.5] and [4] leave $(c_k)_k$ δ -far except with probability $(\theta + \Sigma - 1)W/|\mathbb{F}|$; if $(c_k)_k$ is δ -far, the curve bound over λ , of degree $m - 1$, leaves $\sum_k \lambda^k c_k$ δ -far except with probability $(m - 1)W/|\mathbb{F}|$. The union bound gives

$$\varepsilon_{\text{prox}} = (\theta + \Sigma + m - 2) W / |\mathbb{F}|, \quad (5)$$

the single-table error with $U = \Sigma + m - 1$. The query term is unchanged: one code, one width W , t columns. The eval sumcheck adds $2v/|\mathbb{F}|$, and the batching of claims by η , of packed columns by ρ_i and of masters by λ adds terms whose numerators are linear in Σ , m and c , without the factor W . The per-table ZeroCheck and LogUp terms do not change.

Both parties compute the shape from absorbed data in integer arithmetic and reject one whose query or proximity term falls below 100 bits; the verifier holds the LogUp term, 106 to 128 bits on the workloads below, to the same floor. Table 1 lists the shape of each workload of Section 8 in zero-knowledge mode. The proximity term is the smaller on every row, 102 to 108 bits; the query term is 109 to 118.

workload	T	v	c	$\log_2 W$	U	query	proximity
ML-KEM-768 sender	9	13	13	14	229	114	106
ML-KEM-768 receiver	8	15	14	15	206	116	105
ML-DSA-44	7	15	14	15	226	116	105
ML-DSA-65	7	15	14	15	272	116	104
ML-DSA-87	7	16	15	16	239	117	104
AES-128	3	19	16	17	194	118	103
AES-256	3	19	16	17	234	118	103
RSA-2048	3	15	15	16	143	117	104
SHA-256, 2^{11} rows	1	12	12	13	91	109	108
SHA-256, 2^{21} rows	1	21	17	18	219	118	102
Keccak-f, 2^{15} rows	1	15	14	15	84	116	106
Keccak-f, 2^{20} rows	1	20	16	17	84	118	104
Fibonacci, 2^{20} rows	1	20	15	16	12	117	108
Fibonacci, 2^{24} rows	1	24	17	18	12	118	105
Fibonacci, 2^{26} rows	1	26	18	19	12	118	104

Table 1: Shape and security bits of the pool in zero-knowledge mode, as the verifier logs them, $t = 287$, $S = 288$. $U = \Sigma + m - 1$; the proximity column is $128 - \log_2 W - \lceil \log_2(v - c + U - 1) \rceil$.

6 The split

The split c minimizes an integer estimate of the bytes that depend on it:

$$\text{cost}(c) = 17 \cdot 2^c + t \sum_i 2^{\max(n_i - c, 0)} \ell_i(P_i) + 32K \sum_i (P_i - 1). \quad (6)$$

The first term is the data part of q , priced at 17 bytes per element, a one-byte bincode varint tag and 16 bytes; its support part, S elements from $c = 11$, is not priced. The second is the opened symbols: $\ell_i(P)$ is the byte count of table i 's symbols on one grid row of one column, both trees, at P columns per slot. The third is the claims on blind columns: each of the KP_i blind columns of table i sends two masked claims of 16 bytes, and the term counts the $K(P_i - 1)$ blinds beyond those of an unpacked table. In base mode $K = 0$.

The search runs over $1 \leq c \leq \max_i(n_i + p_i)$, p_i the packing depth of table i 's widest class, and admits c when it leaves room for the support, $2^c \geq S$, and when the shape of Section 5 clears 100 bits. Ties go to the narrower split. The inputs are the absorbed heights, the program and the configuration; nothing in the proof sets c .

Not counted: the Merkle multiproof, at most t hashes per tree per unit of c , and the eval sumcheck, $2v$ elements; the masters and the remaining claims do not depend on c . With both added, the minimizer moves on no workload of Table 1 in either mode, and under that count the next split costs at least 18.6 KiB more.

The third term matters for small tables: a table of 2^4 rows in a pool at $c = 16$ packs $P = 2^{12}$, and its 4096 blind columns send 128 KiB of claims and 8192 masked scalars to the outer argument. Every table of Table 1 has at least 2^9 rows, and the term stays below 4 KiB per table there.

7 Leaves and the support

Unsalted leaves need one support cell more than the query count. In zero-knowledge mode the verifier sees the fold vector q , data and support parts, the masters, and the opened

symbols of at most t distinct columns. Every other column reaches it only through its Merkle leaf, a hash with no salt. If an unopened column is a function of the witness and the view, a guess of the witness can be tested against the root.

Lemma 3. *Let $S = t$. For every message (ξ, d) and every column j , the symbol at j is an affine function of d and the symbols at the t opened columns.*

Proof. The symbol at a point α is $\sum_{k < S} \xi_k X_k(\alpha) + \sum_{k < L} d_k X_{S+k}(\alpha)$. Since $\deg X_k = k$, the X_k with $k < S$ span the polynomials of degree below S , and a nonzero one has fewer than S roots. The $S \times t$ matrix $(X_k(\alpha_j))$ over the opened points is therefore invertible at $S = t$, and ξ is an affine function of d and the opened symbols. Substituting it gives the symbol at j . $\square$

At $S = t$, then, a tree whose rows carry no uniform data cell has every unopened leaf fixed by the witness and the view. The h tree is such a tree: it carries no blind. The trace tree of a pool with one table, without ring units, on one grid row is another: there q fixes the data of the one blind slot as q 's data part minus the witness terms, and Lemma 3 fixes the rest.

Lemma 4. *Let $S \geq t + 1$, and let at least two messages carry nonzero weight in q . Given the view and every data cell, the symbol of any message at any unopened column is uniform.*

Proof. With the data fixed, the view is linear in the supports ξ_r . Put $x_j = (X_k(\alpha_j))_{k < S}$ and let A be the span of $x_{j'}$ over the opened columns j' . Polynomials of degree below $S \geq t + 1$ interpolate any $t + 1$ values, whence $x_j \notin A$ for an unopened j . The view determines $\langle \xi_r, a \rangle$ for every r and $a \in A$, and $\sum_r w_r \xi_r$, w the weights of q . Were $\langle \xi_r, x_j \rangle$ determined as well, it would equal $\sum_{r'} \langle \xi_{r'}, a_{r'} \rangle + \langle \sum_{r'} w_{r'} \xi_{r'}, u \rangle$ identically, every $a_{r'} \in A$. A message $r' \neq r$ with $w_{r'} \neq 0$ forces $u \in A$, and then $x_j = a_r + w_r u \in A$, a contradiction. $\square$

In the pool every table has a data slot and a blind slot, and each weight of (3) is a nonzero polynomial of degree below $\Sigma + m + \theta$ in the challenges: the hypothesis fails with probability at most $2(\Sigma + m + \theta)/|\mathbb{F}|$.

Table 2 checks both lemmas, and the hiding of the whole linear view, by exact ranks over $\mathbb{F}$ on the deployed layout code. Every entry of the view and every unopened symbol is a linear form over masking cells, supports and blinds, and witness cells, trace and h ; with B the masking part of the view and A its witness part, the view is independent of the witness if and only if $\text{rank}(B) = \text{rank}(B \mid A)$, and a leaf is fixed when the masking part of every symbol it hashes lies in the row space of B . The pools sit at $c = 4$, $W = 32$ and $t = 6$ distinct queries, leaving 26 unopened columns: three tables of 2^6 , 2^2 and 2^1 rows, the first on four grid rows and the others packed, two with ring units, two with LogUp columns and each with one blind; and one table of 2^2 rows packed on one grid row. The computation first checks the line (4) and the encoding of q against the folded columns.

The linear view hides the witness in all four configurations, and both controls register their leak. At $S = t$ every unopened leaf of the h tree is fixed, and of the trace tree too when one table sits on one grid row; at $S = t + 1$ none is. The deployed configuration takes $S = t + 1 = 288$, and both parties reject $S \leq t$, at the cost of one element of q . A salt per leaf would also free every leaf, at t salts per opening.

8 Measurements

On ML-KEM-768, ML-DSA and AES the pool changes proof size by -36 to -56% , and on RSA-2048 by -88% . We measured the example workloads of Hekate [12] on an Apple M3

pool	S	$\text{rank}(B)$	$\text{rank}(B \mid A)$	fixed leaves	
				trace	h
three tables	t	206	206	0/26	26/26
three tables	$t + 1$	207	207	0/26	0/26
one table, one grid row	t	34	34	26/26	26/26
one table, one grid row	$t + 1$	35	35	0/26	0/26
second table without blind	$t + 1$	200	201		
three tables	$t - 1$	185	205		

Table 2: Exact ranks over $\mathbb{F}$ of the view at $c = 4$, $W = 32$, $t = 6$. The last two rows are controls with a planted leak.

workload	T	proof, KiB		verify, ms		prove, ms		peak, MiB
		separate	pool	separate	pool	separate	pool	pool
ML-KEM-768 sender	9	3 508	1 544	72.1	11.6	567	408	204
ML-KEM-768 receiver	8	3 902	1 884	88.6	13.9	754	552	292
ML-DSA-44	7	2 867	1 547	50.7	10.5	549	432	219
ML-DSA-65	7	2 959	1 568	52.0	10.3	594	472	207
ML-DSA-87	7	3 336	1 812	55.7	11.2	766	618	308
AES-128	3	4 665	2 970	20.1	6.4	1 230	920	770
AES-256	3	4 960	3 198	19.9	6.5	1 330	975	961
RSA-2048	3	14 724	1 810	29.4	8.2	334	204	397
SHA-256, 2^{11} rows	1	540	516	9.4	3.2	69	53	62
SHA-256, 2^{21} rows	1	5 485	5 485	22.7	11.3	10 650	8 720	5 099
Keccak-f, 2^{15} rows	1	858	851	11.9	4.6	195	163	143
Keccak-f, 2^{20} rows	1	3 542	3 544	17.1	7.1	3 730	3 230	2 430
Fibonacci, 2^{20} rows	1	1 146	1 143	6.8	3.7	328	259	252
Fibonacci, 2^{24} rows	1	4 098	4 098	12.9	8.1	5 420	4 060	3 321
Fibonacci, 2^{26} rows	1	8 022	8 021	21.3	12.4	23 320	19 270	13 003

Table 3: Zero-knowledge mode, best of three runs; separate commitments in 0.36.0 at $S = t$, the pool in 0.37.0 at $S = t + 1$.

Max, with the verifier built with whole-program link-time optimization and one codegen unit; each figure is the best of three runs. Peak memory is the larger of the peak physical footprint and the maximal resident set. The pool runs at $S = t + 1$, the configuration of Table 1; the separate commitments run at $S = t$, to which $S = t + 1$ would add 16 bytes per table.¹ Table 3 collects the zero-knowledge figures.

Every RSA-2048 table has at most 2^{10} rows, and the pool at $c = 15$ packs at least 32 of its columns per slot; no AES table has fewer than 2^{16} rows, and AES gains from the shared fold vector and query set alone. Base mode carries no blinds and no outer argument, and its figures are not tabulated: proof size changes by -37 to -67% on the same seven, and by -90% on RSA-2048. On the seven single-table workloads proof size changes by -0.8 to $+0.1\%$, except SHA-256 at 2^{11} rows, 540 to 516 KiB, while verification time changes by -37 to -66% and prove time by -13 to -25% through changes outside the pool.

¹Hekate 0.36.0 with prover 0.14.0 for the separate commitments, 0.37.0 with prover 0.15.0 for the pool; signed public builds 6f09b831 and 4e72a3c1.

9 Remarks

Lemmas 3 and 4 concern the leaves. The hiding of the linear view rests on the blinds and the outer argument [10], and Table 2 checks it on instances only.

Prove time depends on how the prover schedules encoding and hashing, which this note does not describe.

References

- [1] Scott Ames, Carmit Hazay, Yuval Ishai, and Muthuramakrishnan Venkitasubramanian. Liger: lightweight sublinear arguments without a trusted setup. *Designs, Codes and Cryptography*, 2023.
- [2] Eli Ben-Sasson, Dan Carmon, Yuval Ishai, Swastik Kopparty, and Shubhangi Saraf. Proximity gaps for Reed–Solomon codes. Cryptology ePrint Archive, Paper 2020/654, 2020. FOCS 2020.
- [3] David G. Cantor. On arithmetical algorithms over finite fields. *Journal of Combinatorial Theory, Series A*, 50(2):285–300, 1989.
- [4] Benjamin E. Diamond and Angus Gruen. Proximity gaps in interleaved codes. Cryptology ePrint Archive, Paper 2024/1351, 2024. IACR Communications in Cryptology, 2025.
- [5] Benjamin E. Diamond and Jim Posen. Polylogarithmic proofs for multilinear over binary towers. Cryptology ePrint Archive, Paper 2024/504, 2024.
- [6] Stephen Duan. Ziren: Succinct arguments for MIPS32 execution. Cryptology ePrint Archive, Paper 2026/2330, 2026.
- [7] Alexander Golovnev, Jonathan Lee, Srinath Setty, Justin Thaler, and Riad S. Wahby. Brakedown: Linear-time and field-agnostic SNARKs for R1CS. Cryptology ePrint Archive, Paper 2021/1043, 2021. CRYPTO 2023.
- [8] Ulrich Haböck. Multivariate lookups based on logarithmic derivatives. Cryptology ePrint Archive, Paper 2022/1530, 2022.
- [9] Tamir Hemo, Kevin Jue, Eugene Rabinovich, Gyumin Roh, and Ron D. Rothblum. Jagged polynomial commitments (or: How to stack multilinear). Cryptology ePrint Archive, Paper 2025/917, 2025. EUROCRYPT 2026.
- [10] Andrei Kochergin. Zero-knowledge ring-switching without the transpose. <https://github.com/oumuamua-labs/hekate/blob/902ab6735348/docs/zk-ring-switching.pdf>, 2026.
- [11] Sian-Jheng Lin, Wei-Ho Chung, and Yung-Hsiang S. Han. Novel polynomial basis and its application to Reed–Solomon erasure codes. In *55th IEEE Annual Symposium on Foundations of Computer Science*, pages 316–325, 2014.
- [12] Oumuamua Labs. Hekate. Source repository <https://github.com/oumuamua-labs/hekate>, 2026.
- [13] The Plonky3 Authors. Plonky3. Source repository <https://github.com/Plonky3/Plonky3>, commit a21f9e3, 2026. Mixed matrix commitment scheme, trait `Mmcs`.